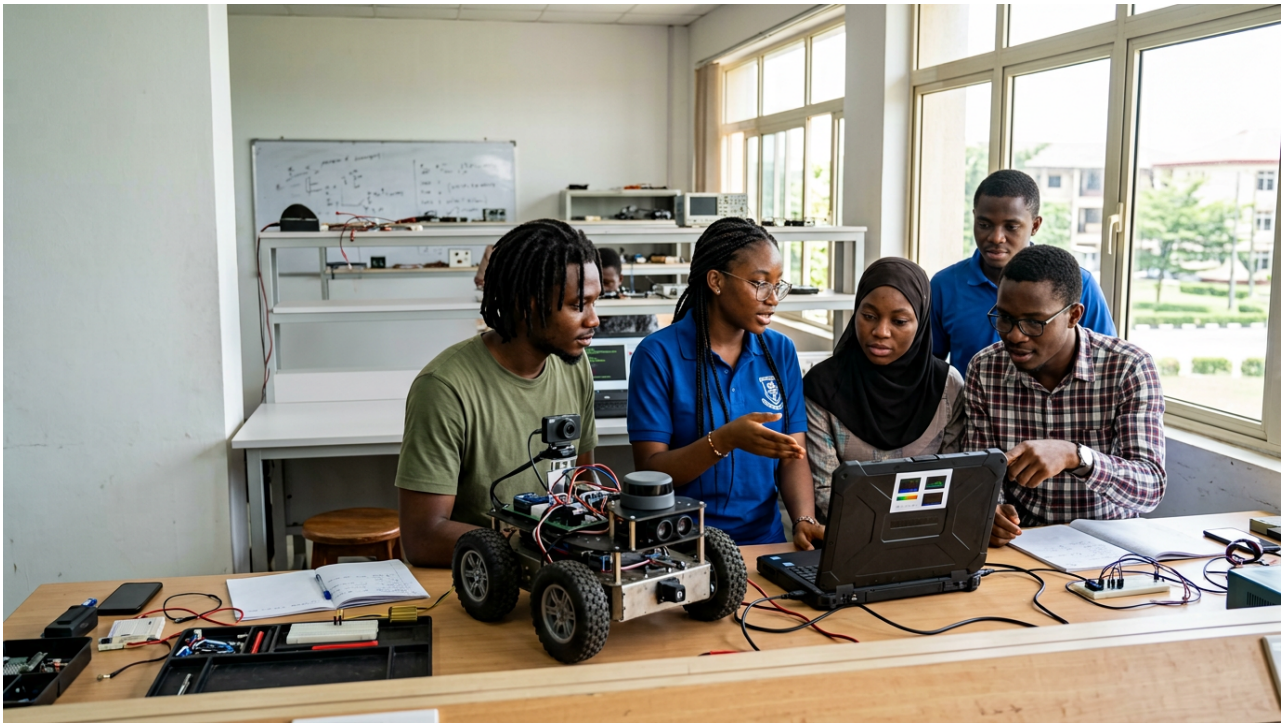
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 8: Neural Networks and Generative AI

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Neural networks as learnable functions

Machine learning includes many kinds of models. Neural networks are one family, built from layers of computational units connected by adjustable parameters. The name is inspired by biological nervous systems, but an artificial neural network is a mathematical and computational construction. It should not be confused with a complete model of a brain.

A neural network maps inputs to outputs. A small network for motor-temperature prediction might take current, speed and ambient temperature as inputs and produce a temperature estimate. A vision network may take pixel values and produce object scores. A language network may process a sequence of tokens and estimate what token or sequence is likely next. The network's parameters, often called weights and biases, determine the mapping.

A simple unit combines input values with weights, adds a bias, and passes the result through an activation function. A layer applies many such operations. Multiple layers can build intermediate representations. In image processing, early layers may respond to edges or textures, while later layers may respond to combinations associated with objects. These interpretations are useful intuitions, but real networks can be more complex and their internal representations are not always easy to interpret.

Show three input values feeding weighted connections into a hidden layer, then an output. Animate one weighted sum and activation at a time. Label the network as a function with parameters.]

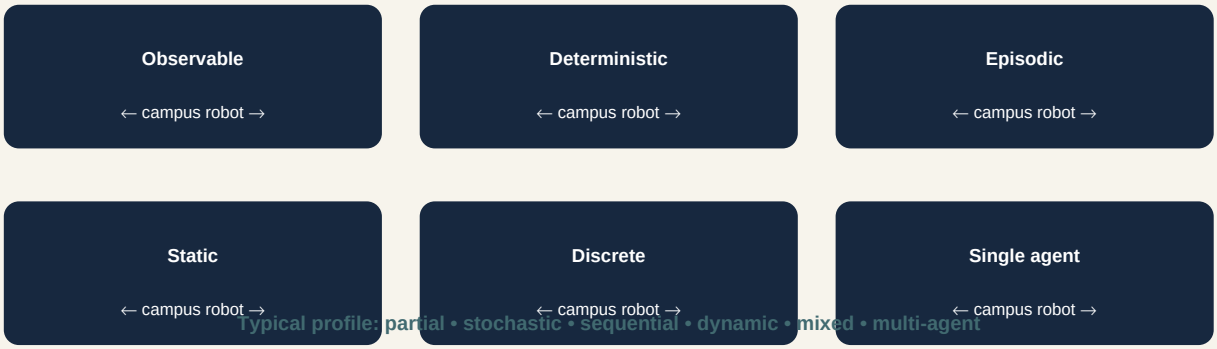


Diagram: Environment classification profile

2. Training and error correction

During training, the model produces an output for an example. A **loss function** measures how different that output is from a target or desired outcome. An optimization method adjusts parameters to reduce loss across training examples. In many networks, gradient-based methods use derivatives to estimate how each parameter contributes to the loss. Backpropagation efficiently computes these gradients through layers.

At an introductory level, the idea is: make a prediction, measure error, determine how the parameters influenced that error, and adjust them slightly in a direction that is expected to reduce future error. Training repeats this process over examples and batches. An **epoch** is one complete pass through the training set, though practical training schedules vary.

Loss reduction is not the same as task success. The selected loss may not align perfectly with real consequences. A model trained to minimize average pixel error might produce visually smooth images while omitting a safety-critical detail. A classifier trained on an imbalanced dataset may obtain low average loss while performing poorly on a rare class. Evaluation must examine behavior relevant to the application, not just whether the training objective decreases.

The learning rate controls the size of parameter updates. If it is too large, training may oscillate or diverge. If too small, learning may be slow or get stuck in an unhelpful region. Batch size, initialization, normalization, regularization and data quality can all influence results. These choices are engineering parameters, not magic switches.

3. Why depth can help—and complicate matters

A multilayer network can represent complex functions by composing simpler transformations. This is one reason deep learning has been effective for images, audio and text. Instead of manually defining every visual feature, a network can learn useful representations from examples.

But larger networks generally require careful data management and significant computation. They may be hard to interpret, can memorize biases in the data, and can be sensitive to changes in the environment. A network that recognizes a tool in a clean workshop image may fail under glare, motion blur or an unfamiliar camera. A network that produces fluent language may generate unsupported claims. Greater capacity expands what a model can fit; it does not guarantee that the fitted behavior is robust or appropriate.

Convolutional neural networks use operations that exploit spatial structure in images. Recurrent networks and other sequence models were developed to process ordered data. Transformers use attention mechanisms to relate elements in a sequence or other structured input. These architectures are important, but the course will treat them as tools with assumptions and trade-offs, not as universal solutions.

4. Generative models

A generative model learns patterns in data and can produce new samples. Depending on the model, those samples may be text, images, audio, code or other structured outputs. A large language model is trained to model patterns in sequences of tokens and commonly produces text by estimating likely continuations given context.

The ability to generate a coherent continuation is not the same as verifying that a statement is true. A model may produce an answer that is well formed and plausible while unsupported or incorrect. It may also reflect biases, gaps or contradictions in its training data. A fluent style can create an unwarranted impression of certainty. Treat generated content as output to evaluate, not as a source of guaranteed facts.

A generative system’s behavior depends on the prompt, the model, decoding settings, context limits, retrieval or tool connections, and other system components. The same prompt can produce different responses. If factual grounding is needed, designers may connect the model to a curated knowledge source, show citations or evidence, constrain outputs, and require human review. These measures can reduce some failures but do not eliminate the need for testing.

5. Transformers and attention at a high level

A transformer processes token sequences using layers that allow each token representation to incorporate information from other tokens. An attention mechanism computes which parts of the context are relevant to a given representation under learned parameters. In language models, this supports relationships across words and phrases that may be far apart in the sequence.

The system does not simply search a dictionary for a fixed answer. It computes representations and produces likely output tokens in sequence. Tokenization matters: a word may be divided into pieces, and the model’s context capacity is measured in tokens rather than words. The context window limits what information can be considered at one time, and long prompts may require selection or summarization.

A short sentence is split into tokens. Arrows in an attention map connect tokens to relevant context; then a probability distribution over next tokens is shown. Avoid implying that attention weights are a complete explanation of model reasoning.

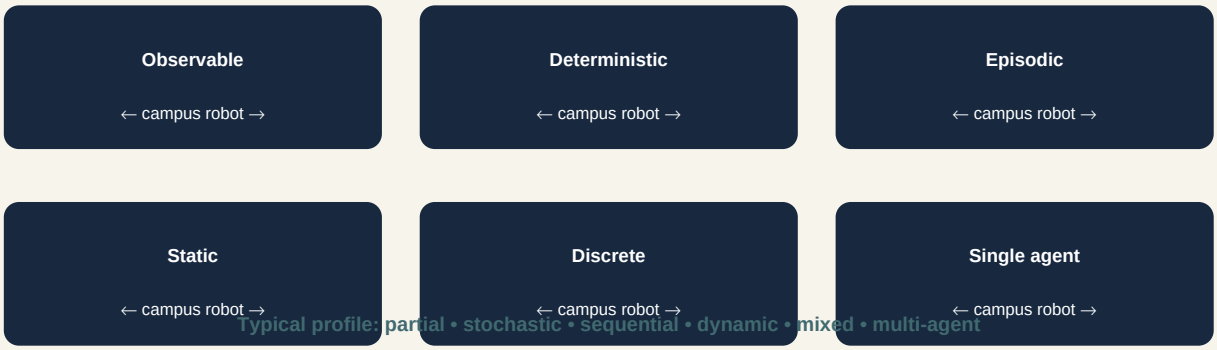


Diagram: Environment classification profile

Attention is an important mechanism, but visualizing attention does not automatically explain why a model produced a particular answer. Interpretability requires additional methods and careful claims. Model explanations can be incomplete, unstable or misleading if presented as definitive causal accounts.

6. A bounded application in robotics

Imagine a maintenance robot that allows a technician to ask questions about an equipment manual. A language model might help find relevant passages and summarize a troubleshooting procedure. The robot's physical control should not be delegated to free-form generated text. Instead, the assistant can retrieve approved instructions, cite the source section, present uncertainty, and ask the technician to confirm before any consequential action.

A safe architecture separates functions. The perception subsystem detects the equipment and reads identifiers. A retrieval component searches approved documents. A language model drafts a concise explanation from those documents. A verification layer checks that cited passages exist and that the response follows allowed formats. The human technician remains responsible for confirming the diagnosis and authorizing physical work. If the manual is unavailable or the evidence is ambiguous, the system says so rather than fabricating a procedure.

This example illustrates a general pattern: use generative AI for language flexibility where it helps, but keep action authority, safety constraints, verification and accountability explicit. A generative model can be one component in a larger engineered system; it is not automatically the system's controller, truth source or safety case.

7. Evaluation and responsible deployment

Evaluate neural models on data that reflect intended operating conditions. For a vision model, vary lighting, angle, camera quality, object condition and background. For a language assistant, test ambiguous prompts, missing information, conflicting sources, adversarial instructions and requests outside its scope. Record not only successes but also failure types and severity.

Check subgroup performance where relevant. A face or speech system can perform differently across populations, recording conditions or accents. A system intended for broad use should be tested across the people and environments it affects. Avoid collecting sensitive data without a clear need and appropriate safeguards. Consider retention, access, consent and exposure through prompts or logs.

Deployment also requires compute and energy planning. Large models can require specialized hardware, introduce latency and increase cost. A smaller model, retrieval system, symbolic rule or standard program may be more appropriate. Consider whether the application can run locally, whether the system must work offline, and how updates will be validated.

For any model integrated with a robot, monitor inputs and outputs. Detect missing sensors, out-of-distribution observations and invalid responses. Define a safe fallback. A model score alone is not a safety argument. The complete system requires testing, constraints, documentation and clear responsibility for oversight.

8. Practical lab: build and evaluate a small neural model

Use a modest dataset appropriate for an introductory exercise, such as classifying simple manufactured-part images or estimating a numeric sensor output. First implement a baseline, such as a linear model or a small decision tree. Then train a small neural network using a standard framework. Keep training, validation and test data separate.

Plot the training and validation loss over epochs. Identify whether the model is underfitting, overfitting or still improving. Try one regularization technique, such as early stopping or weight decay, and compare results. Inspect misclassified examples rather than reporting only a single accuracy value. If images are used, test a change in brightness or orientation and report how performance shifts.

For a generative-AI extension, build a narrow study assistant that answers questions from a provided short technical document. Require the system to quote or cite the relevant passage and to state when the document does not contain an answer. Create tests for answerable, unanswerable and contradictory questions. Have a human verify every cited claim. Do not connect the prototype to a real robot’s motor commands.

9. Common mistakes

One mistake is to equate a neural network with intelligence as a whole. A network is a model component; the full system includes data, input pipelines, objectives, interfaces and operation.

Another mistake is to interpret a fluent generated response as verified knowledge. Fluency measures style, not truth.

A third mistake is to treat training loss as proof of useful behavior. Held-out testing and scenario-based evaluation are essential.

A fourth mistake is to assume that more parameters always produce a better engineering choice. Cost, latency, privacy, interpretability and maintenance matter.

A fifth mistake is to allow generative text to issue physical actions without constraints, confirmation or safety checks. Separate explanation from authority to act.

10. Knowledge check and assignment

What is a neural network’s role in a machine-learning system? How do weights change during training? Why can training loss improve while real-world performance remains poor? Why is a plausible generated answer not necessarily correct? What responsibilities belong to components around a language model in a robotic application?

For the assignment, train and evaluate a small neural model against a baseline. Report data splits, model size, training curves, metrics, failure cases and a deployment constraint. Then design a one-page architecture for a bounded generative-AI assistant. Identify its data sources, verification mechanism, human oversight, privacy risks and safe behavior when evidence is insufficient.

Module summary

Neural networks are parameterized functions whose weights are adjusted during training to improve an objective. Their layered representations have enabled strong performance in many domains, while also introducing data, compute, evaluation and interpretability challenges. Generative models produce plausible samples but do not guarantee factual correctness. A responsible engineering design treats neural or generative models as components, evaluates them under relevant conditions, separates language generation from action authority, and provides verification and fallback behavior.